

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5

B)5.0

C)25

D)sqrt(25)

Q2. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```

```
print(b)
```

A)[10]

[10]

B)[10, 10]

[10]

C>Error

D)[10]

[10, 10]

Q3. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)24

B)10

C)0

D)9

Q4. What will be printed?

```
def make_counter():
    count = 0
```

```
def counter():
    nonlocal count
    count += 1
    return count
```

```
return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

1

1

1

B)1

2

1

3

C)1

2

3

4

D)0

1

0

2

Q5. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

A)TypeError



B)It executes correctly

C)SyntaxError

D)NameError

Q6. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

A)Nothing

B)A

C)B

D)A B

Q7. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

try:

```
    test()
```

except MyError as e:

```
    print(type(e).__name__)
```

A)Exception

B)TypeError

C)Something went wrong

D)MyError

Q8. What happens here?

try:

```
    raise ValueError("Wrong value")
```

except ValueError as e:

```
    print(e)
```

A)Nothing

B)e

C)ValueError

D)Wrong Value

Q9. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:
```

return 20

print(fun())

A)20

B)None

C)RuntimeError

D)10

Q10. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

**codes's second handler never
reached?**

A)ZeroDivisionError is not an exception

B)ZeroDivisionError occurs before
Exception

C)Exception is a superclass of
ZeroDivisionError

D)except blocks execute randomly

**Q11. What is the purpose of this
statement?**

```
if __name__ == "__main__":
```

A)It identifies code intended to run when
the file is executed directly

B)It creates the main function

C)It prevents all imports

D)It prevents syntax errors

Q12. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

print(fun(3))

A)5

B)6

C)3

D)None

Q13. What is the output?

try:

```
    x = int("abc")
```

except ValueError:



```

    print("A")
else:
    print("B")
finally:
    print("C")

```

- A)B C
- B)B
- C)A
- D)A C

Q14. What is the output?

```

try:
    raise ValueError
except ValueError:
    print("A")
    raise

```

- A)B
- B)A, and execution ends normally
- C)No Output
- D)A, followed by propagation of the same exception

Q15. What will be printed?

```

try:
    x = int("10")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")

```

- A)B C
- B)A C
- C)B
- D)A

Q16. What happens here?

```

from math import sqrt

```

```

print(sqrt(36))

```

- A)6.0
- B)36
- C)6
- D)NameError

Q17. What will be printed?

```

try:
    print(10 / 0)

```

```

except ZeroDivisionError:
    print("A")
except Exception:
    print("B")

```

- A)A
- B)A B
- C)Nothing
- D)B

Q18. if test.py has print("A")

```

if __name__ == "__main__":
    print("B")

```

If test.py is executed directly, what will be printed?

- A)A followed by B
- B)A
- C)B
- D)Nothing

Q19. What is the output?

```

def fun(x):
    x += 10
    return x

```

```

x = 5
y = fun(x)

```

```

print(x, y)

```

- A)15 15
- B)5 15
- C)15 5
- D)5 5

Q20. What will be the output?

```

x = 10

```

```

def outer():
    x = 20

```

```

    def inner():
        global x
        x = 30

```

```

    inner()
    print(x)

```

```

outer()

```



print(x)

A)30

20

B)20

30

C)30

30

D)20

10

Q21. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

print(result)

A)[2, 4, 6, 8, 10]

B)[4, 8]

C)[2, 4]

D)[1, 3, 5]

Q22. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

A)5

B)func(5)

C>Error

D)10

Q23. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
        return x
    except ValueError:
        return 100
    finally:
        print("cleanup")
```

print(fun(-5))

A)100

cleanup

B)cleanup

100

C)cleanup

-5

D)negative

cleanup

Q24. What is the major difference between the following?

import math

and

from math import sqrt

A)There is no difference

B)Both import only sqrt

C)First provides the module namespace; second imports sqrt directly

D)First imports only sqrt

Q25. What will be printed?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        print(x)
```

```
    x = 30
    inner()
```

```
outer()
```

A)20

B)NameError

C)30

D)10

Q26. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

A)20



B)30

C)TypeError

D)SyntaxError

Q27. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

A)20

B)NameError

C)10

D)UnboundLocalError

Q28. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

A)5

B)15

C)10

D)UnboundLocalError

Q29. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)Python will always reject it

B)Python automatically renames the local file

C)The local file can shadow the intended standard-library module

D)random cannot be imported

Q30. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)None

B>Error

C>Hello None

D>Hello

Q31. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()
```

A)SyntaxError

B)NameError

C)10

D)20

Q32. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)RecursionError

B)3

C)TypeError

D)6

Q33. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        return x + 5
```

```
    return inner
```



```
f = outer()
```

```
print(f())
```

A)NameError

B)None

C)10

D)15

Q34. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)A B

B)A

C)SyntaxError

D)B

Q35. What happens when this code runs?

```
def first():
```

```
    second()
```

```
def second():
```

```
    third()
```

```
def third():
```

```
    raise RuntimeError("Failure")
```

```
try:
```

```
    first()
```

```
except RuntimeError:
```

```
    print("Handled")
```

A)RuntimeError cannot propagate through functions

B)Handled

C)TypeError

D)NameError

Q36. What will happen?

```
def fun(a, b=10):
```

```
    return a + b
```

```
print(fun(b=5, a=20))
```

A)15

B)25

C)SyntaxError

D)TypeError

Q37. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)0

B)NameError

C)TypeError

D)None

Q38. What is the output?

```
def fun():
```

```
    try:
```

```
        print("A")
```

```
        raise ValueError
```

```
    except ValueError:
```

```
        print("B")
```

```
        return 10
```

```
    else:
```

```
        print("C")
```

```
        return 20
```

```
    finally:
```

```
        print("D")
```

```
print(fun())
```

A)A

C

D

20

B)A

B

D

10

C)A

B

10

D

D)A

B

C

D

10

Q39. Which object keeps track of modules that have already been



imported during the current Python process?

- A)sys.imports
- B)sys.modules
- C)__module__
- D)sys.path

Q40. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)20
- B)-10
- C)30
- D)10

Q41. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)B
- B)A
- C)Nothing
- D)A and B

Q42. What will normally happen?

```
import math
```

```
import math
```

- A)The module is normally loaded and executed twice
- B)ImportError
- C)The module is normally loaded once and reused
- D)SyntaxError

Q43. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:
```

```
print("Finally")
```

```
print(fun())
```

- A)10 followed by Finally
- B)10 only
- C)Finally followed by 10
- D)Finally only

Q44. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

- A)Only Hello
- B>Error
- C)x
- display
- D)10
- Hello

Q45. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)10
- 10
- B)ImportError
- C)Module loaded
- Module loaded
- 10



D)Module loaded

10

Q46. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)NameError

B)UnboundLocalError

C)50

D)10

Q47. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q48. What is the output?

f = lambda x: x + 5

print(f(10))

A)5

B>Error

C)10

D)15

Q49. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

A)NameError

B)None

C)10

D)20

Q50. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

A)TypeError

B)30

C)SyntaxError

D)20

